



TÉCNICO
LISBOA

Organização de Computadores

LEIC-T

2025-2026

Relatório Lab. 2 (Turno OC2112L02)

ist1110179
ist1109881
ist1109605

Renato Filipe Antunes Neto
Tomás Martins Ferreira
Sebastião Gomes Sequeira Lemos de Mendonça

Objetivo/(breve introdução):

O objetivo deste projeto é implementar uma Translation Lookaside Buffer (TLB) em que permita realizar de forma mais eficiente a tradução de endereços virtuais em endereços físicos. O input do programa baseia-se em dois tipos de operações: Read e Write (R "virtual_address" e W "virtual_address"). Cada uma destas chamadas invoca a função **tlb_translate**, implementada por nós, que tem como objetivo procurar o endereço físico na TLB (ou, em caso de falha, na Page Table).

A nossa implementação segue o formato **NINE** (*Non-Inclusive, Non-Exclusive*), onde os dois níveis (L1 e L2) funcionam de forma independente. Nesta concretização L1 e L2 (os dois níveis da TLB) funcionam de forma independente, uma entrada pode estar em ambos ou em apenas um.

Esta implementação permite reduzir penalizações de coerência, ou seja, não é necessário atualizar ou invalidar entradas noutro nível sempre que ocorre uma alteração (exceto no caso de um **tlb_invalidate** explícito do endereço).

Pontos importantes desta implementação:

- Em caso de **miss na L1 e hit na L2**, o endereço deve ser colocado na L1. Se esta estiver cheia, aplica-se o método **LRU (Least Recently Used)**.
- Em caso de **miss na L1 e L2**, o endereço deve ser adicionado a **ambas** as TLBs, seguindo novamente o método LRU se uma ou ambas estiverem cheias.
- Em caso de **hit na L1**, apenas é necessário retornar o endereço físico (*physical address*).

Implementação:

- Tlb Invalidate

A função **tlb invalidate** serve para remover (invalidar) entradas nos caches TLB L1 e L2 que correspondem ao virtual page number dado.

A função **tlb invalidate** percorre os dois níveis da TLB (L1 e L2) e limpa o bit de validade correspondente à página virtual especificada, incrementando o contador de invalidação do nível relevante.

O objetivo é garantir a coerência do sistema de memória durante o Despejo de Páginas (Page Eviction). Quando o Sistema Operacional remove uma página virtual da memória física (DRAM) como ao salvá-la no disco, a tradução antiga na TLB torna-se incorreta.

Ao invalidar a entrada, força o sistema a consultar a Tabela de Páginas na próxima vez, carregando a informação correta e atualizada.

- Tlb Translate

A função **tlb_translate()** tem como objetivo traduzir um endereço virtual (virtual address) num endereço físico (physical address). Para isso, a função simula o funcionamento de uma TLB (Translation Lookaside Buffer) que contém dois níveis, L1 e L2.

A função recebe o **virtual_page_number** e o **op** (tipo de operação) e começa por percorrer o nível L1 da TLB à procura da página referente ao endereço virtual. Se ela estiver em L1, atualizamos o bit dirty para 1, caso a operação seja write, o **last_access** para 0 e as restantes páginas incrementamos o valor de **last_access**. Neste caso houve um *hit* em L1, e é retornada a respetiva tradução do endereço virtual.

Em caso de *miss* em L1 é percorrido o nível L2 da TLB com o mesmo processo, no entanto sem retornar logo a tradução. Caso esta procura resulte num *hit* em L2, o nível L1 é percorrido até encontrar posições livres (valid bit igual a zero) registando o índice de inserção em L1 (**insert_l1**). Se não forem encontradas posições livres, registamos o índice da página com o maior valor de **last_access** (política de substituição LRU). Posteriormente, a posição do nível L1 correspondente a esse índice é substituída pela página encontrada em L2, tendo em conta que, caso a entrada substituída tenha o bit dirty ativo, este é propagado para a respetiva página no próximo nível (L2), garantido que as modificações efetuadas não sejam perdidas.

No caso da pesquisa no nível L2 da TLB também resultar em *miss*, ambos os níveis (L1 e L2) são percorridos à procura de posições livres, ou seja, entradas com o bit valid a zero. Caso todas as entradas estejam ocupadas, aplica-se a política de substituição LRU (Least Recently Used), selecionando a entrada que foi acedida há mais tempo. Destas pesquisas resultam os índices de inserção para cada nível (**insert_l1** e **insert_l2**). Antes de inserir as novas páginas nos respectivos níveis, é necessário verificar se as entradas a serem substituídas estão marcadas com o bit dirty. Se tal acontecer, o bit é propagado para o próximo nível (L2) ou, no caso do L2, é feito um write-back para a Page Table (PT), de forma a garantir a consistência dos dados entre todos os níveis de memória.

No final, as respetivas páginas são inseridas nas posições determinadas do nível L1 ou, no caso de ocorrer um *miss* em ambos os níveis, são colocadas simultaneamente nos níveis L1 e L2. Por fim, é retornado o endereço físico resultante da tradução.

Considerações Finais:

O desenvolvimento, neste projeto, de uma TLB permitiu compreender de forma prática o tanto o funcionamento como também a importância das hierarquias de memória na tradução de endereços virtuais para endereços físicos. Através da criação de dois níveis de TLB e a utilização de uma política **NINE** (*Non-Inclusive, Non-Exclusive*), foi possível explorar o impacto da organização e da gestão das entradas em diferentes níveis de cache.